

Authentication & Identity Flows

Complete Auth Architecture with Supporting Diagrams

A comprehensive technical reference documenting every authentication and authorisation flow in the Azure AI Agent Architecture — from end-user OAuth2/OIDC through APIM JWT validation, Managed Identity service-to-service flows, AKS Workload Identity OIDC federation, Key Vault secret injection, and Zero Trust defence-in-depth. Includes eight annotated architecture diagrams.

Document Type	Technical Reference — Security & Identity
Scope	All auth flows in the Azure AI Agent Architecture
Platform	Microsoft Entra ID · Azure APIM · ACA · AKS · Key Vault
Protocols	OAuth 2.0 · OIDC · Managed Identity · PKCE · JWT · IMDS
Phase Coverage	Phase 1 (Foundation) → Phase 3 (Enterprise)
Date	April 2026

01 Identity Architecture Overview

The Azure AI Agent Architecture uses Microsoft Entra ID as the single, unified identity plane for all authentication and authorisation — across user-facing APIs, service-to-service calls, infrastructure access, and secret management. This implements a strict Zero Trust model: every request is verified, every identity is granted least privilege, and no credentials are ever stored in application code, environment variables, or container images.

Four Authentication Patterns

- ① OAuth 2.0 Auth Code + PKCE — User-facing**
Interactive authentication for human users via browser. Used for: developer portal, admin interfaces, HIL approval flows. PKCE prevents auth code interception attacks.
- ② Client Credentials Flow — Daemon/Service**
Machine-to-machine for background services with no user context. Used for: Command Workers calling APIM, scheduled tasks, Harness Delegate → Azure operations. No scp claim — uses roles claim instead.
- ③ Managed Identity (IMDS) — Azure Resources**
Azure-native service identity — no credentials anywhere. Used for: Orchestrator → Key Vault, Worker → Azure OpenAI, Delegate → ACR, ACA → Cosmos DB. Azure rotates credentials automatically.
- ④ Workload Identity (OIDC) — AKS Pods**
Kubernetes-native OIDC federation for pods. Used for: Harness Delegate on AKS, Chaos Agent pods. Service Account token exchanged for Entra ID JWT without IMDS interception.

Master Identity Architecture Diagram

The diagram below shows all four authentication patterns in context, with Microsoft Entra ID as the central trust anchor:

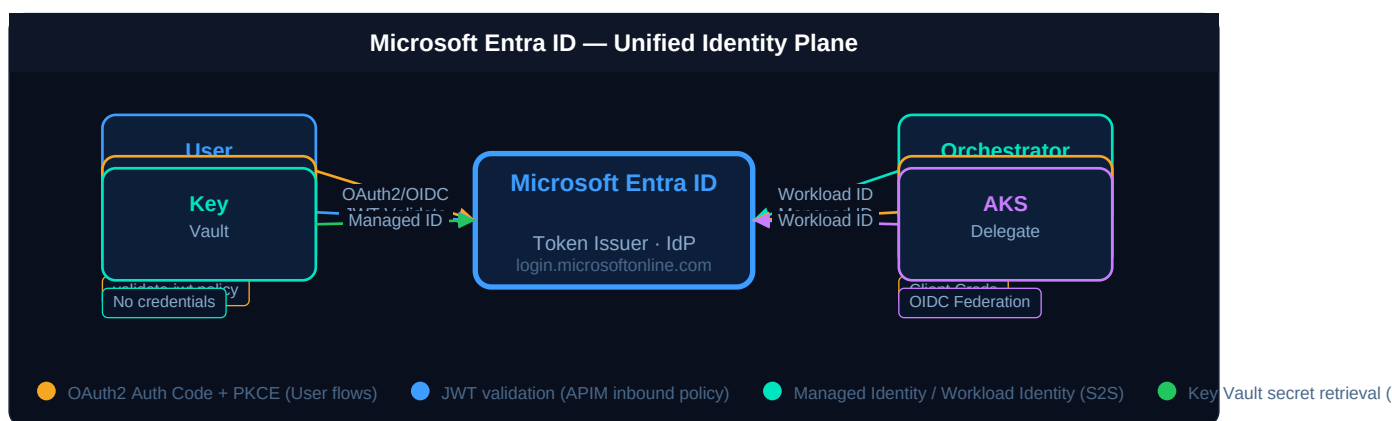


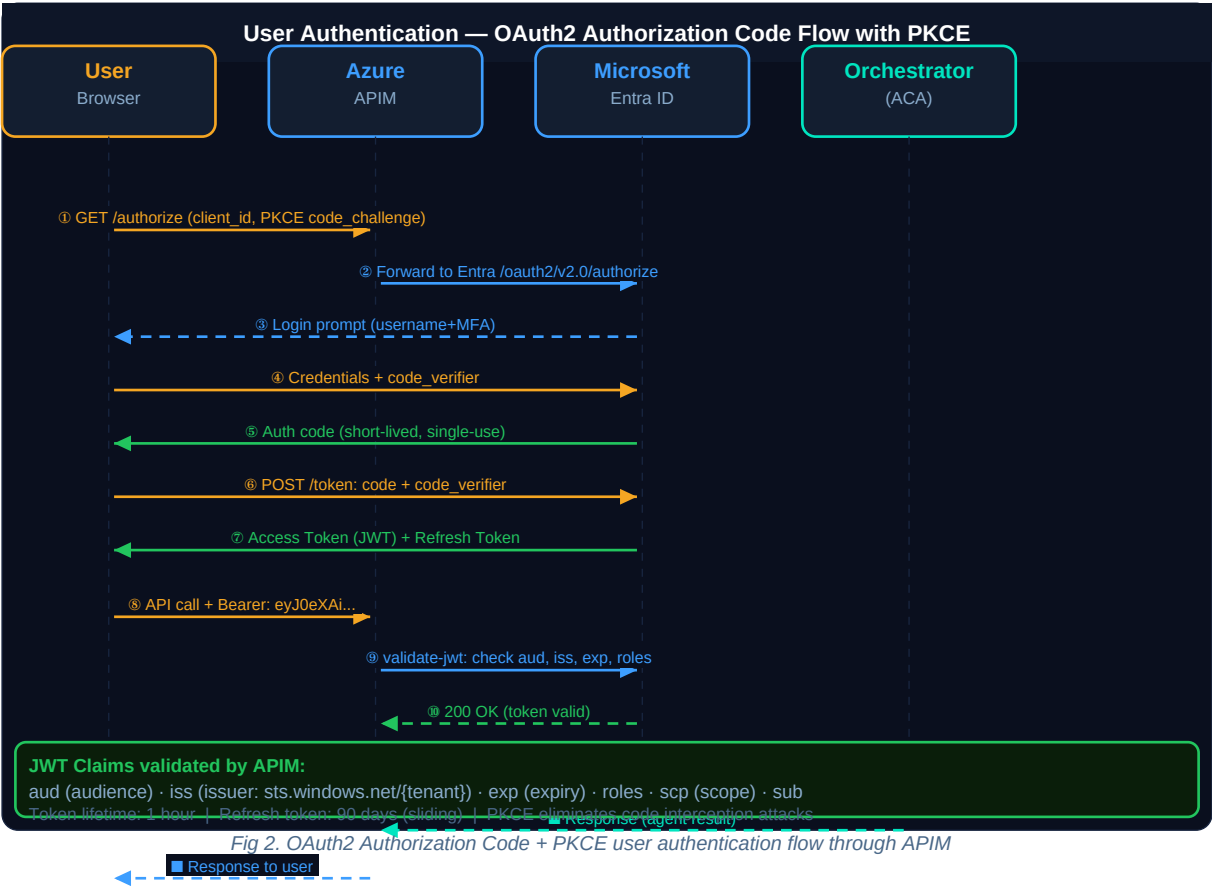
Fig 1. Unified Microsoft Entra ID identity plane with all four auth patterns

02

User Authentication — OAuth2 Auth Code + PKCE

Users accessing the AI Agent system authenticate via the OAuth 2.0 Authorization Code flow with PKCE (Proof Key for Code Exchange). PKCE is mandatory — it eliminates the risk of auth code interception attacks by binding the code exchange to a cryptographic verifier known only to the original requester.

Step-by-Step Flow



Technical Details

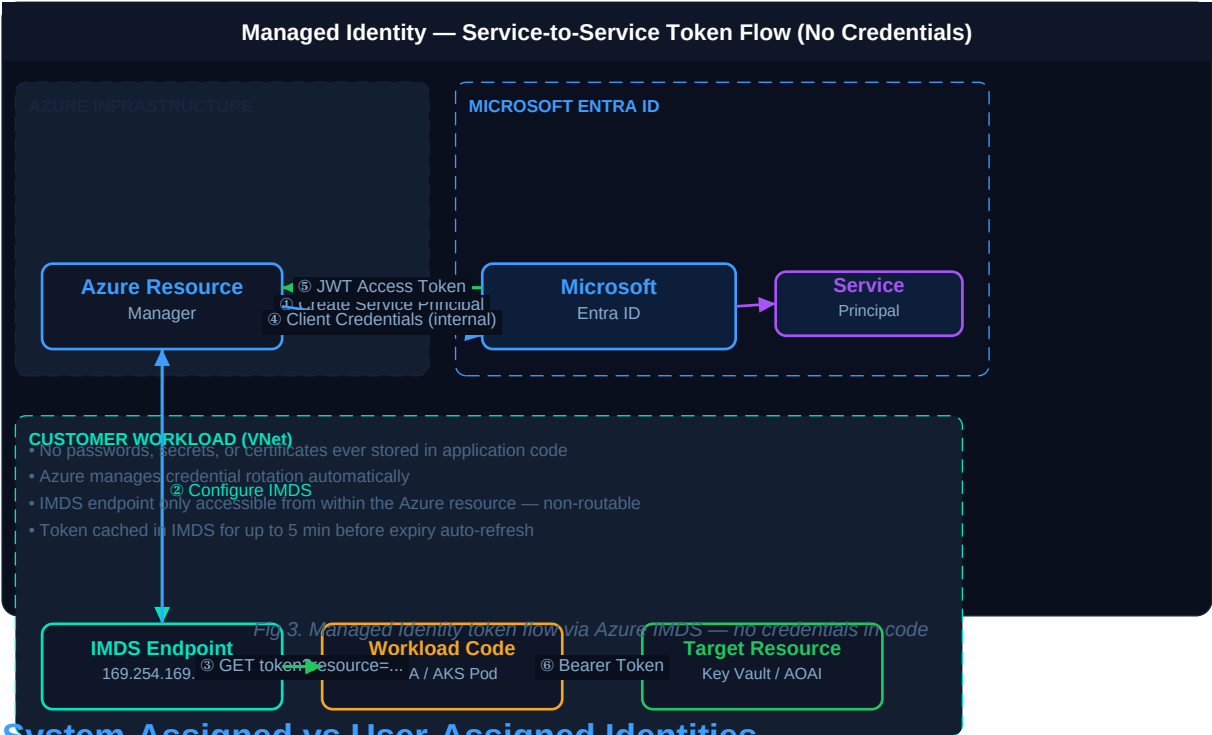
Parameter	Value	Purpose
response_type	code	Authorization code flow (not implicit)
code_challenge_method	S256	SHA-256 hashed PKCE verifier
scope	openid profile api://{app-id}/.default	Request ID token + API access
redirect_uri	https://{apim-host}/.auth/callback	Only registered URIs accepted
token_endpoint	login.microsoftonline.com/{tenant}/oauth2/v2.0/token	Token exchange endpoint

access_token TTL	1 hour (3600s)	Short-lived; refresh token used after
refresh_token TTL	90 days (sliding)	Rotated on every use
MFA enforcement	Conditional Access Policy	Required for all user logins

03 Managed Identity — Service-to-Service (IMDS)

Managed Identity is the primary authentication mechanism for all service-to-service communication within the Azure AI Agent Architecture. When enabled on an Azure resource (ACA, AKS, Functions), Azure automatically creates and manages a Service Principal in Entra ID. The workload obtains tokens from the Azure Instance Metadata Service (IMDS) endpoint — accessible only from within the Azure resource — without storing any credentials.

IMDS Token Flow Diagram



System-Assigned vs User-Assigned Identities

Property	System-Assigned	User-Assigned
Lifecycle	Tied to resource — deleted with resource	Independent — persists after resource deletion
Sharing	Cannot be shared — 1:1 with resource	Can be shared across multiple resources
Use case in AI Agent Arch	Orchestrator, Workers (ephemeral ACA Jobs)	Shared agents, cross-service access patterns
RBAC assignment	On resource creation	Pre-configured; attached on deploy
AKS support	Cluster identity	Workload Identity (preferred for pods)

IMDS Request Example (Python — azure-identity SDK)

```
from azure.identity import DefaultAzureCredential
from azure.keyvault.secrets import SecretClient

# DefaultAzureCredential automatically uses IMDS when running on Azure
```

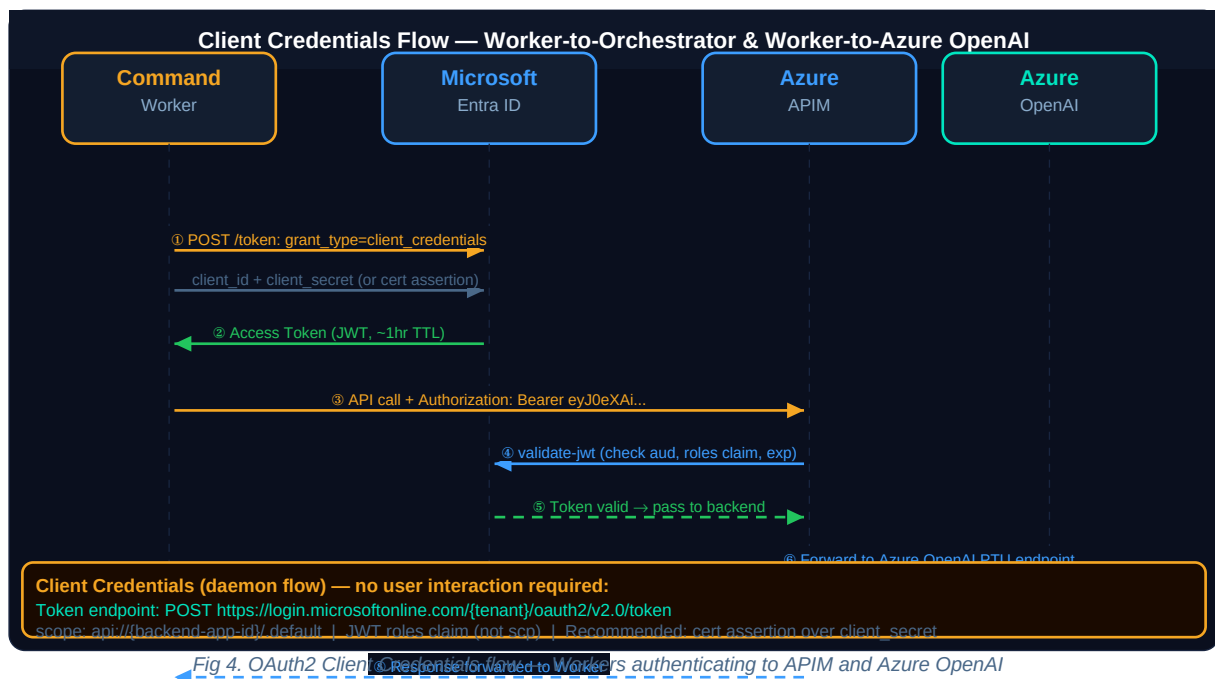
```
credential = DefaultAzureCredential()  
client = SecretClient(vault_url="https://{vault}.vault.azure.net", credential=credential)  
secret = client.get_secret("aoai-api-key")  
# No credentials in code. Token fetched from http://169.254.169.254/metadata/identity/...
```

04

Client Credentials Flow — Workers & Background Services

The OAuth 2.0 Client Credentials grant is used for daemon processes and background services that operate without a signed-in user. In the AI Agent Architecture, Command Workers use this flow to authenticate against APIM when calling Azure OpenAI through the Model Router. Unlike delegated flows, tokens contain a **roles** claim (not scp) and require explicit App Role assignment.

Client Credentials Flow Diagram



Token Request (curl example)

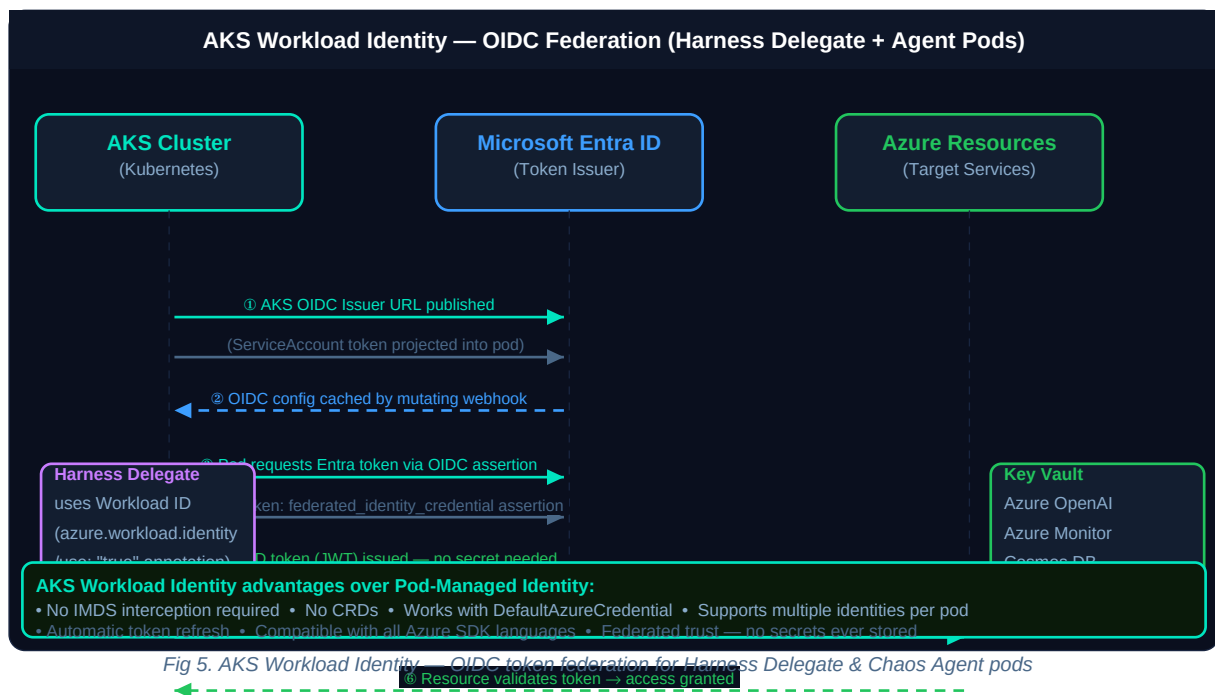
```
# Step 1: Acquire token from Entra ID
curl -X POST \
  "https://login.microsoftonline.com/{tenant-id}/oauth2/v2.0/token" \
  -d "client_id={worker-app-client-id}" \
  -d "client_secret={worker-app-secret}" \
  -d "scope=api://{apim-backend-app-id}/.default" \
  -d "grant_type=client_credentials"

# Step 2: Call APIM with Bearer token
curl -X POST "https://{apim-host}/agents/v1/complete" \
  -H "Authorization: Bearer eyJ0eXAiOiJKV1Q..." \
  -H "task_type: reasoning" \
  -d '{"prompt": "...", "max_tokens": 2000}'
```

Production recommendation: Replace client_secret with a certificate assertion (client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer). Certificate private key stored in Azure Key Vault. APIM's validate-jwt policy checks the **roles** claim (e.g. Worker.Invoke) rather than scp.

05 AKS Workload Identity — OIDC Federation

For workloads running on AKS (primarily the Harness Delegate and Chaos Agent), Microsoft Entra Workload ID uses OIDC federation to issue Azure AD tokens to Kubernetes pods without any IMDS proxy or Custom Resource Definitions. The AKS cluster acts as an OIDC token issuer; service account tokens are projected into pods and exchanged for Entra ID access tokens.



Configuration Requirements

- AKS cluster: Enable OIDC issuer (--enable-oidc-issuer) and Workload Identity webhook (--enable-workload-identity)
- Service Account: Annotate with azure.workload.identity/client-id: {managed-identity-client-id}
- Pod spec: Label with azure.workload.identity/use: "true" (Fail-Close mode for reliability)
- Federated credential: Register OIDC issuer URL + namespace/service-account in Managed Identity
- Code: Use DefaultAzureCredential() — automatically selects WorkloadIdentityCredential on AKS
- Maximum 20 federated credentials per Managed Identity

Harness Delegate Workload Identity YAML annotation:

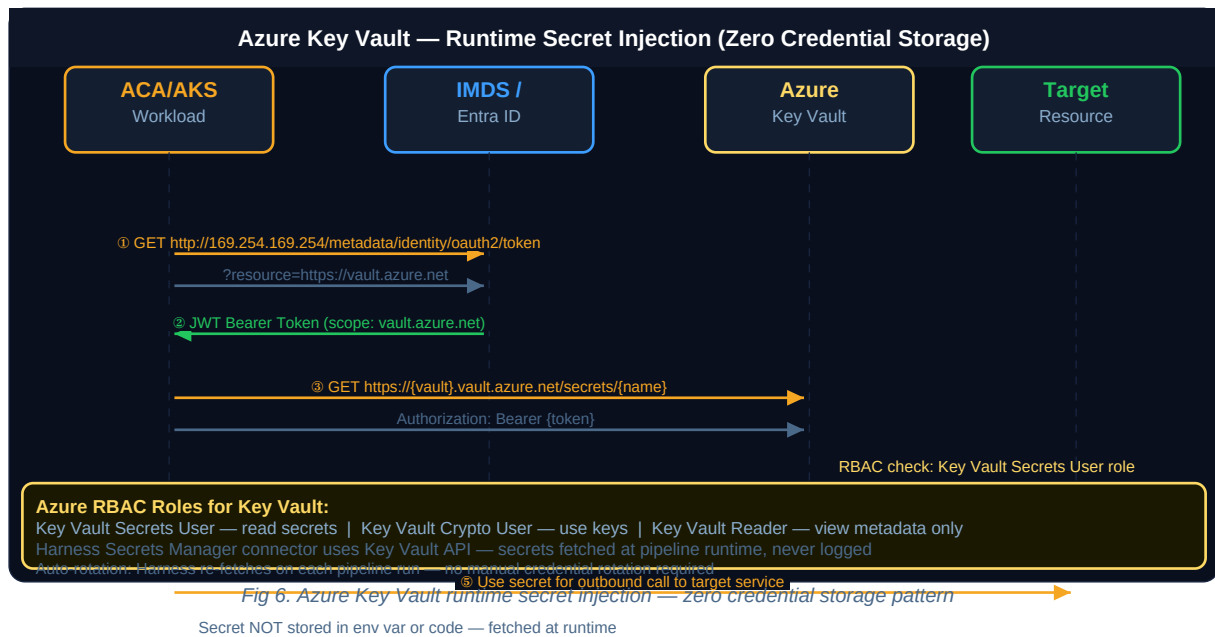
```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: harness-delegate-sa
  namespace: harness-delegate-ng
  annotations:
    azure.workload.identity/client-id: {managed-identity-client-id}
---
```



```
# Pod template label (in Delegate StatefulSet spec):  
labels:  
  azure.workload.identity/use: "true"
```

06 Azure Key Vault — Runtime Secret Injection

Azure Key Vault is the single secret store for the entire AI Agent Architecture. No connection strings, API keys, certificates, or passwords are stored in environment variables, Kubernetes secrets, pipeline YAML, or application code. All secrets are fetched at runtime via Managed Identity using the Key Vault REST API, with access controlled by Azure RBAC.



Secrets Stored in Key Vault

Secret Name	Type	Consumer	Rotation
aoai-api-key-primary	API Key	Orchestrator, Workers via APIM	Monthly auto-rotate
aoai-api-key-secondary	API Key	APIM fallback backend	Monthly auto-rotate
cosmos-connection-string	Connection String	LTM service (Orchestrator)	Quarterly
redis-access-key	Password	STM clients (Orchestrator, Workers)	Quarterly
service-bus-connection	Connection String	Worker Registry (ACR publish)	Quarterly
ai-search-admin-key	API Key	Knowledge Base indexer (ADF)	Quarterly
harness-delegate-cert	Certificate	Harness Delegate (Client assertion)	Annual (auto)
teams-webhook-url	URL Secret	HIL Logic Apps notification	On change
worker-contract-signing-key	HMAC Key	Worker Registry contract validation	Annual

07 APIM JWT Validation — Inbound Policy

Azure API Management acts as the single authorisation gateway for all LLM calls. Every inbound request must present a valid Bearer token. APIM's validate-jwt policy performs cryptographic signature verification, claim validation, and role/scope checking before any request reaches backend services. This enforces authorisation even if backend services do not implement their own auth.

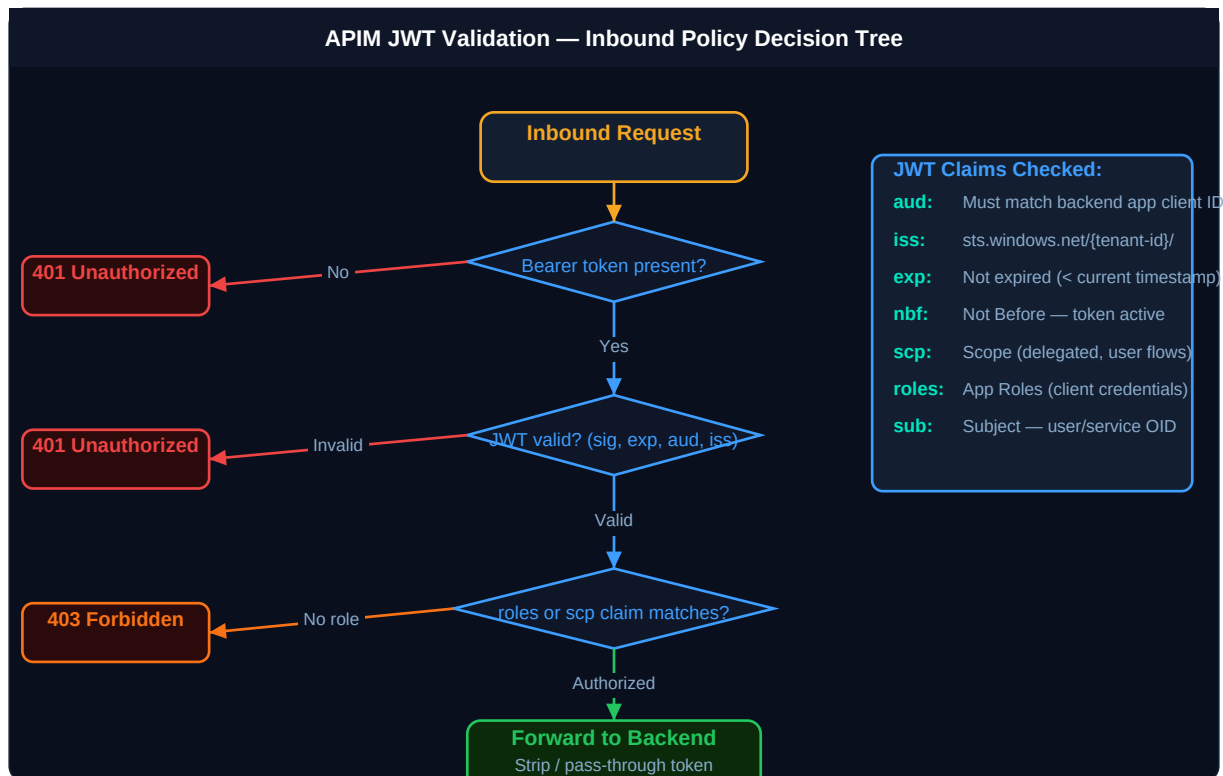


Fig 7. APIM JWT validation decision tree — request authorisation policy flow

APIM validate-jwt Policy (Production Configuration)

```

<policies>
<inbound>
<base />
<validate-jwt header-name="Authorization"
failed-validation-httpcode="401"
failed-validation-error-message="Unauthorized. Token missing or invalid.">
<openid-config url="https://login.microsoftonline.com/{tenant}/v2.0
/.well-known/openid-configuration" />
<audiences>
<audience>api://{backend-app-client-id}</audience>
</audiences>
<issuers>
<issuer>https://sts.windows.net/{tenant-id}</issuer>
<issuer>https://login.microsoftonline.com/{tenant-id}/v2.0</issuer>
</issuers>
  
```

```
<required-claims>
<!-- For client credentials: check roles -->
<claim name="roles" match="any">
<value>Worker.Invoke</value>
<value>Orchestrator.Call</value>
</claim>
</required-claims>
</validate-jwt>
<!-- Strip token before forwarding to backend -->
<set-header name="Authorization" exists-action="delete" />
<!-- Add Managed Identity token for backend auth -->
<authentication-managed-identity resource="https://cognitiveservices.azure.com" />
</inbound>
</policies>
```

08

Zero Trust Architecture Defence in Depth

The AI Agent Architecture is designed with concentric layers. Every service boundary is a security boundary. Lateral movement is restricted to the least privilege scope.

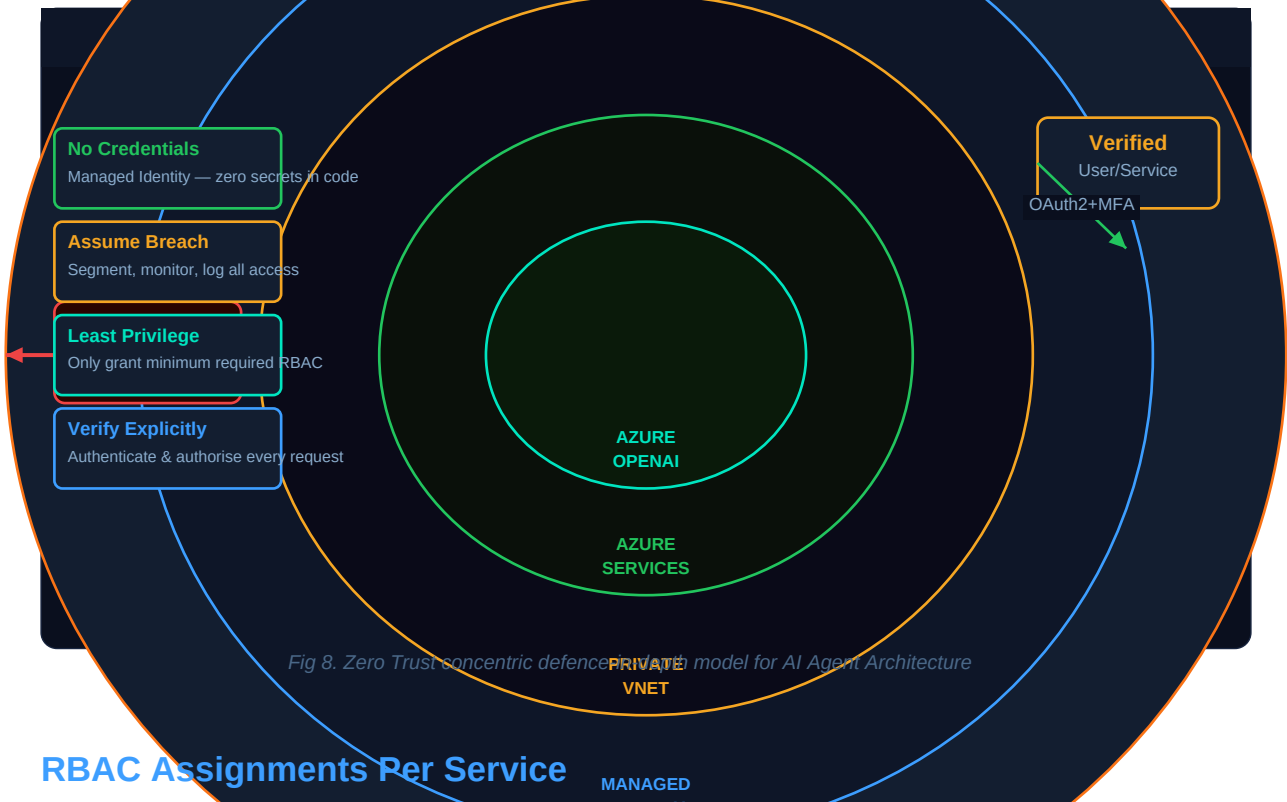


Fig 8. Zero Trust concentric defence model for AI Agent Architecture

RBAC Assignments Per Service

Service	Managed Identity	Azure RBAC Roles Granted
Orchestrator (ACA)	System-Assigned	Key Vault Secrets User · Azure Service Bus Data Sender · Redis Cache Contributor
Command Workers	User-Assigned (shared)	Key Vault Secrets User · Cognitive Services User (via APIM)
MCP Tools (Functions)	System-Assigned	Key Vault Secrets User · Storage Blob Data Reader
Harness Delegate (AKS)	Workload Identity	Contributor (deployment scope) · ACR Push/Pull · Key Vault Admin
Logic Apps HIL	System-Assigned	Service Bus Data Owner · Key Vault Secrets User
ADF (AI Search refresh)	System-Assigned	Storage Blob Data Reader · Search Index Data Contributor
AI Foundry	System-Assigned	Log Analytics Contributor · App Insights Component Contributor

09

Network Isolation — Private Endpoints & VNet

All PaaS services in the AI Agent Architecture are accessed exclusively via Azure Private Endpoints inside a dedicated VNet. No service is exposed to the public internet from within the workload. Egress from ACA and AKS workloads is routed through Azure Firewall with FQDN allow-lists. This prevents token leakage, SSRF attacks, and data exfiltration.

Network Security Controls

Control	Implementation	Auth Benefit
Private Endpoints	Azure Private Link on all PaaS (AOAI, Cosmos, Key Vault, Service Bus, etc.)	Endpoints only valid within VNet — no internet exposure
Azure Firewall	Egress via Firewall with FQDN allow-list (login.microsoftonline.com, vault.azure.com)	Prevents SSRF and unauthorized token requests
NSG Rules	Deny-all default; explicit allow inbound from APIM subnet only	Prevents lateral movement between subnets
UDR (Route Tables)	Force tunnel to Azure Firewall for all egress	Intercepts and inspects all outbound auth traffic
APIM Internal Mode	APIM deployed in internal VNet mode — no public IP	JWT validation enforced before traffic enters VNet
Conditional Access	Require MFA + compliant device for admin identities	Prevents credential stuffing on admin accounts

09.1 Conditional Access Policies

Conditional Access enforces contextual authorisation on top of token validation:

- Require MFA for all user-facing authentication (Orchestrator portal, HIL approval, admin console)
- Require Azure AD-joined or Intune-compliant device for admin/ops roles
- Block legacy authentication protocols (basic auth, NTLM) — OAuth2/OIDC only
- Named locations: allow only corporate IP ranges for admin role access
- Sign-in risk policy: Block sign-ins with high Entra ID Identity Protection risk score
- Token binding: Enable Continuous Access Evaluation (CAE) for near-real-time revocation

10

Implementation Checklist & Common Pitfalls

10.1

Implementation Checklist

Phase 1 — Identity Foundation

- Register backend-app in Entra ID; expose API URI (api://{client-id}); define App Roles
- Register worker-app; grant application permissions (not delegated); admin consent
- Enable System-Assigned Managed Identity on all ACA environments
- Enable AKS OIDC Issuer + Workload Identity webhook on AKS cluster
- Configure Key Vault with RBAC authorization (not vault access policies)
- Assign Key Vault Secrets User role to each Managed Identity
- Deploy APIM validate-jwt policy on all API operations
- Configure APIM Authentication header stripping (delete Authorization before backend)
- Enable Entra ID Conditional Access: require MFA for user flows

Phase 2 — Scale & Hardening

- Implement Continuous Access Evaluation (CAE) for real-time token revocation
- Replace client_secret with certificate assertion for all Worker identities
- Enable Azure Defender for Key Vault — alert on anomalous secret access
- Configure Key Vault auto-rotation for AOAI keys (Event Grid trigger on near-expiry)
- Set up User-Assigned Managed Identity for Worker pool (shared identity)
- Annotate all AKS workloads with Workload Identity labels
- Enable APIM caching of OIDC configuration — reduce token validation latency

Phase 3 — Enterprise Governance

- Enable Privileged Identity Management (PIM) for all administrative roles
- Configure Entra ID Identity Protection: risk-based Conditional Access
- Implement access reviews: quarterly review of all App Role assignments
- Enable Entra ID audit logs → Azure Monitor → Log Analytics for SIEM integration
- Configure Azure Policy: deny creation of secrets in code, enforce Managed Identity use
- Red-team exercise: token theft simulation, SSRF testing via Harness PyRIT
- Implement just-in-time (JIT) access for Key Vault admin operations via PIM

10.2

Common Pitfalls & Mitigations

Pitfall	Risk	Mitigation
Storing client_secret in env vars	Secret leakage via logs/config dumps	Use Managed Identity or cert assertion; Key Vault ref
Wrong audience (aud) in token	401 errors on every request	Use api://{backend-app-id}/.default scope exactly
scp vs roles confusion	Authorization always fails for daemon apps	Client credentials → roles claim; delegated → scp claim

IMDS endpoint behind proxy	Token acquisition fails silently	IMDS (169.254.169.254) must be non-proxied; configure exclusion
Token not cached locally	IMDS throttling (429) under load	Use DefaultAzureCredential — built-in token cache (5 min)
Missing federated credential	Workload Identity pod fails auth	Register OIDC issuer URL + namespace/SA in Managed Identity
Validate-jwt missing in APIM	Backend auth skipped entirely	Policy must be in inbound section; test with expired token
Broad RBAC roles (Contributor)	Excessive privilege — blast radius	Use minimum built-in roles (Secrets User, not Admin)

References: [Microsoft Learn — OAuth 2.0 and OIDC on Microsoft identity platform](#) · [Azure Container Apps Authentication](#) · [Managed Identities for Azure Resources](#) · [AKS Workload Identity Overview](#) · [APIM validate-jwt policy](#) · [Azure Key Vault RBAC](#) · [Conditional Access documentation](#) · [Microsoft Entra ID Zero Trust guidance](#)